

HW2 數位_IC B1121123 巫煊歲

```
module MEM_WRCNTL (  
    MEM_DO,  
    MEM_DI,  
    WRMEM_Done,  
    RDMEM_Done,  
    WENAB,  
    RENAB,  
    CLK,  
    RESET_  
);  
    // -----  
    // I/O signals  
    // -----  
    output [`WL-1:0] MEM_DO;    // data output of memory  
    wire  [`WL-1:0] MEM_DO;  
  
    output          WRMEM_Done; // Done signal for WR and RD  
    output          RDMEM_Done; // Done signal for WR and RD  
    reg             WRMEM_Done;  
    reg             RDMEM_Done;  
  
    input  [`WL-1:0] MEM_DI;    // data input of memory  
    input          WENAB;       // Write enable  
    input          RENAB;       // Read enable  
    input          CLK;         // clock  
    input          RESET_;      // reset
```

```
    // Declaration for internal control signals  
    // -----  
    reg [4:0] ADDR_CNT;  
    reg       ACNT_ENAB;  
    reg       WEN;  
  
    // -----  
    // Design  
    // -----  
  
    // Address Counter  
    always @(posedge CLK or negedge RESET_) begin  
        if (!RESET_)  
            ADDR_CNT <= `DLY 5'd0;  
        else if (ACNT_ENAB)  
            ADDR_CNT <= `DLY ADDR_CNT + 5'd1;  
        else  
            ADDR_CNT <= `DLY 5'd0;  
    end  
  
    // ACNT_ENAB logic  
    always @(*) begin  
        if ((!WENAB || RENAB) && !(ADDR_CNT == 5'd31))  
            ACNT_ENAB = 1'b1;  
        else  
            ACNT_ENAB = 1'b0;  
    end
```

```

// WEN (Write active low, Read active high)
always @(*) begin
    if (!WENAB)
        WEN = 1'b0;
    else
        WEN = 1'b1;
end

// WRMEM_Done
always @(posedge CLK or negedge RESET_) begin
    if (!RESET_)
        WRMEM_Done <= `DLY 1'b0;
    else if (ADDR_CNT == 5'd31 && !WENAB)
        WRMEM_Done <= `DLY 1'b1;
    else
        WRMEM_Done <= `DLY 1'b0;
end

// RDMEM_Done
always @(posedge CLK or negedge RESET_) begin
    if (!RESET_)
        RDMEM_Done <= `DLY 1'b0;
    else if (ADDR_CNT == 5'd31 && RENAB)
        RDMEM_Done <= `DLY 1'b1;
    else
        RDMEM_Done <= `DLY 1'b0;
end

```

```

105 // Register file instantiation
106 // -----
107 `ifdef IC_ENAB
108     RF1_32x24b DUT_MEM (
109         .Q    (MEM_DO),
110         .CLK   (CLK),
111         .CEN   (1'b0),    // 0 表示啟用，1 表示閒置
112         .WEN   (WEN),     // add your wr/rd enable signal
113         .A     (ADDR_CNT), // add your address bus
114         .D     (MEM_DI)
115     );
116 `else
117     // MEM Design for evaluation
118 `endif
119
120 endmodule

```

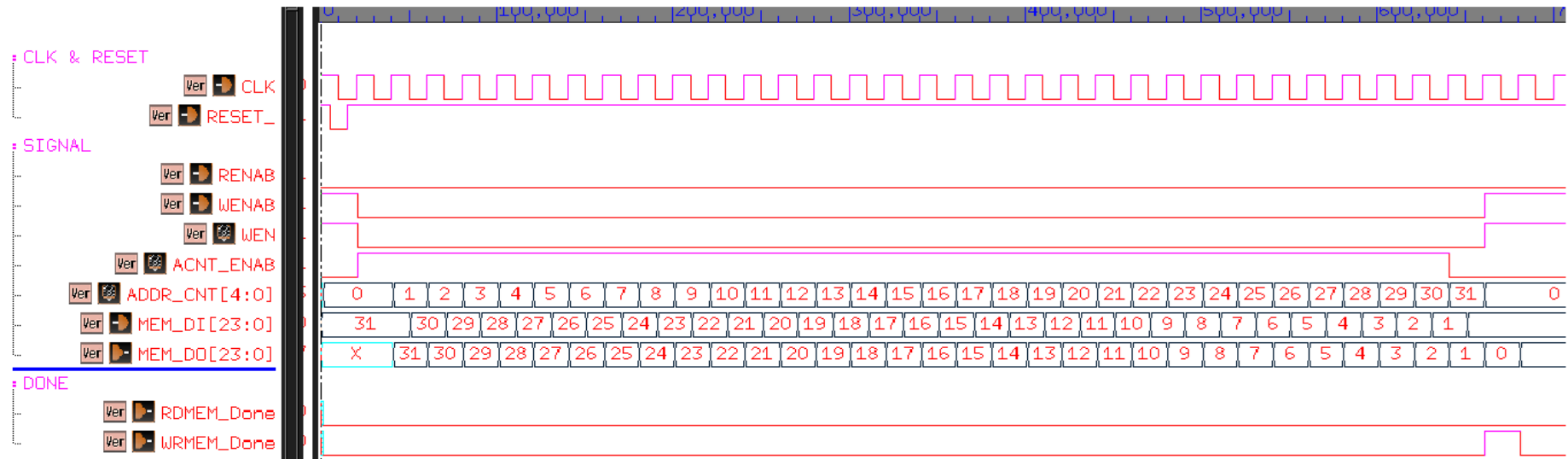
結果

```
FSDB Dumper for VCS, Release Verdi_1-2022.06, Linux x86_64/64bit, 05/29/2022
(C) 1996 - 2022 by Synopsys, Inc.
*Verdi* FSDB WARNING: The FSDB file already exists. Overwriting the FSDB file may crash the programs that are using this file.
*Verdi* : Create FSDB file 'HW2_MEM_WRCNTL.fsdb'
*Verdi* : Begin traversing the scope (MEM_WRCNTL_RTB), layer (0).
*Verdi* : End of traversing.
// ----- Write operation -----
// ----- Read operation -----

Congratulations!! Your Verilog Code is correct!!

$finish called from file "HW2_MEM_WRCNTL_rtb.v", line 111.
$finish at simulation time 1402000
      V C S   S i m u l a t i o n   R e p o r t
Time: 1402000 ps
CPU Time: 0.200 seconds;      Data structure size: 0.0Mb
Sat Apr 25 10:57:24 2026
CPU time: .196 seconds to compile + .203 seconds to elab + .143 seconds to link + .214 seconds in simulation
```

WRITE 波形圖



READ 波形圖

